

Training One Model to Master Cross-Level Agentic Actions via Reinforcement Learning

Kaichen He
Peking University

hkc20@stu.pku.edu.cn

Zihao Wang
Peking University

zhwang@stu.pku.edu.cn

Muyao Li
Peking University

2200017405@stu.pku.edu.cn

Anji Liu
National University of Singapore
anjiliu@comp.nus.edu.sg

Yitao Liang
Peking University
yitaol@pku.edu.cn

Abstract

The paradigm of agentic AI is shifting from engineered complex workflows to post-training native models. However, existing agents are typically confined to static, predefined action spaces—such as exclusively using APIs, GUI events, or robotic commands. This rigidity limits their adaptability in dynamic environments where the optimal granularity of interaction varies contextually. To bridge this gap, we propose CrossHA, a unified agentic model that masters heterogeneous action spaces and autonomously selects the most effective interface for each step of a trajectory. We introduce a comprehensive training pipeline that integrates cold-start supervised fine-tuning with a Multi-Turn Group Relative Policy Optimization (GRPO) algorithm. This approach enables the agent to learn adaptive action switching—balancing high-level efficiency with low-level precision—without human-specified rules. Extensive experiments on over 800 tasks in the open-world Minecraft environment demonstrate that CrossHA achieves state-of-the-art performance. By dynamically leveraging the strengths of diverse action spaces, our model significantly outperforms fixed-action baselines, exhibiting superior generalization and efficiency in long-horizon reasoning. All code and models are available at <https://github.com/CraftJarvis/OpenHA>.

1. Introduction

The field of agentic AI is undergoing a paradigm shift, moving from engineering complex workflows around pre-trained Large Language Models (LLMs) [3, 30, 42, 53] or Vision-Language Models (VLMs) [1, 17, 40, 44] toward developing native agentic models through post-training. Contemporary native agents are typically characterized by the

specific action spaces they master: GUI agents interact via mouse and keyboard events [33, 34], Deep Research agents utilize API calls [20, 32], Tool-Calling agents integrate with Model Context Protocol (MCP) services [16], and Vision-Language-Action (VLA) models execute embodied robotic commands [6, 7, 9]. However, prior research typically necessitated distinct action space designs for different tasks, often requiring manual selection and definition. To enable interaction within these spaces, researchers had to design specific action translation methods—such as implementing MCP servers [2] for function calls or training learning-based policies for embodied control [5].

However, relying on statically defined action spaces presents two fundamental challenges. First, specific action policies or translation layers are often brittle; for instance, an API-based `read_url` function may be blocked by CAPTCHA verifications, or a robotic policy may fail to execute a command with perfect precision, thereby capping the agent’s success rate [5, 14]. Second, the manual assignment of action spaces to tasks restricts the agent’s flexibility, preventing it from effectively handling complex scenarios that require multimodal interactions. Recent works have begun to address this by bridging disjoint spaces to enhance generalization [41]. Specific approaches include integrating GUI and API-based actions into unified models [31] or merging heterogeneous trajectories during training.

Crucially, we observe that the optimal action space often varies not only across tasks but also within a single task at the step level. Consider a Deep Research agent: while the majority of information gathering is efficiently handled via search APIs, navigating a specific CAPTCHA-protected webpage may require precise GUI-level manipulation [32]. Consequently, a truly generalist agent must possess the ability to dynamically switch between action spaces, ranging from high-level APIs to low-level primitives, to maximize both success rates and efficiency.

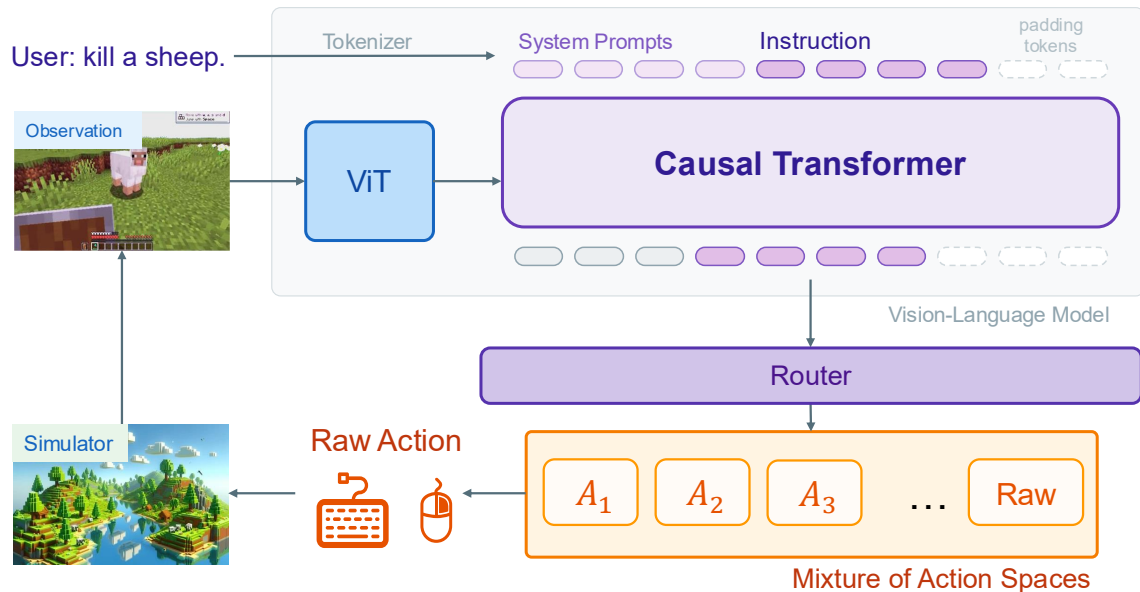


Figure 1. **The CrossHA Framework.** Unlike prior methods that confine the agent to a fixed action space (e.g., atomic movements) throughout a trajectory, CrossHA dynamically switches across different action spaces to adapt to the context.

To address these limitations, we introduce **CrossHA**, a native agentic model trained to master multiple heterogeneous action spaces. Unlike traditional approaches that rely on static, human-specified rules, CrossHA autonomously selects the most appropriate action space for each step of a trajectory. We propose a comprehensive training pipeline comprising three stages: cold-start supervised fine-tuning, Single-Turn Reinforcement Learning (RL), and Multi-Turn RL. By leveraging a Multi-Step Group Relative Policy Optimization (GRPO) [18, 38] algorithm, the agent learns to optimize its choices at the step level without explicit human intervention. This capability enables the agent to balance trade-offs dynamically; for example, prioritizing high-level actions for efficiency when applicable, while employing fine-grained atomic actions for precise control when necessary.

We validate our approach in the open-world Minecraft environment [15] (see [Appendix A](#) for environment details). Experimental results demonstrate that CrossHA, despite being trained on only 30 tasks, successfully generalizes to over 800 tasks [28]. It autonomously selects optimal action spaces, significantly outperforming baselines confined to fixed action spaces. Furthermore, the agent exhibits emergent behavior by optimizing not only for task success but also for trajectory efficiency, demonstrating robust capabilities for long-horizon reasoning.

Our main contributions are summarized as follows: 1) We propose CrossHA, a unified agentic model capable of mastering heterogeneous action spaces and autonomously selecting the context-appropriate interface without relying

on human-defined heuristics. 2) We introduce a comprehensive RL training pipeline utilizing Multi-Turn GRPO, enabling the agent to learn adaptive action switching within a single trajectory to maximize both task success and execution efficiency. 3) We achieve state-of-the-art performance on over 800 tasks in the Minecraft environment, demonstrating that adaptive action-space selection yields superior generalization and robustness compared to static baselines.

2. Related Works

2.1. Agentic Models with Different Actions

The deployment of VLM and LLM-based agents has expanded rapidly across various domains, ranging from physical environments to digital workspaces, each necessitating distinct interaction paradigms. In embodied settings, approaches such as OpenX [29], RT-H [6], and OpenVLA [22] focus on mapping high-level instructions directly to physical control policies, effectively grounding language into continuous robot trajectories or joint angles [8, 14]. Transitioning to the digital domain, research on GUI and Web agents, such as Mind2Web and OS-World, centers on visual grounding, where agents learn to manipulate specific UI elements or generate low-level mouse and keyboard events to navigate dynamic screens [13, 24, 41, 51]. In parallel, for logic-intensive tasks such as software engineering, agents like Claude and SWE-agent operate within discrete symbolic spaces, leveraging shell commands or standardized protocols like MCP to seamlessly integrate with external tools and APIs [2, 3, 16, 48, 54]. Similarly, many VLM-

based game agents interact with video games directly via mouse and keyboard commands [25, 49, 50].

2.2. Generalist Agent with Multi-Action Space

The static design of action spaces creates limitations when facing dynamic environments [49]. A common solution is to leverage the powerful in-context learning capabilities of foundation models [3, 30, 40] to orchestrate workflows that specify transitions between action spaces for specific scenarios. For example, some computer-use agents [33, 39] integrate tool usage [32] with GUI actions to perform complex tasks: utilizing GUI actions for frontend interface validation while employing MCP when information retrieval is required. However, this approach often necessitates the design of complex, brittle pipelines.

A more unified approach involves fine-tuning agents across multiple action spaces to create a single generalist model. Prior explorations have demonstrated promising results in various domains, such as embodied agents [10, 27], game agents [46, 49, 50], and computer use [41]. Experiments have demonstrated [21, 52] that agents mastering distinct action spaces exhibit strong generalization capabilities and enhanced performance within specific domains.

However, previous efforts have often overlooked the potential of learning from experience to optimize these transitions. Our approach builds on prior work by advancing this paradigm, leveraging reinforcement learning to enable the model to autonomously adapt to the most appropriate action space at any given step.

3. Method

In this section, we introduce our approach for building an agent capable of autonomously selecting the most appropriate action space based on the task context.

We first formulate the problem. Consider an embodied control task modeled as a Markov Decision Process (MDP) with a state space $\mathcal{S}$ and a composite action space:

$$\mathcal{A} = \bigcup_{x=1}^N \mathcal{A}_x.$$

Each subspace $\mathcal{A}_x$ corresponds to a distinct class of actions, ranging from low-level motor controls to high-level motion primitives. Each subspace is associated with a specific interface or controller C_x that executes the abstract action $a_t \in \mathcal{A}_x$ within the environment.

At each timestep, the agent must determine **both** the optimal action space and the specific action content, balancing the immediate reward with execution efficiency. This leads to the following objective:

$$J = \mathbb{E} \left[\sum_t (r_t - \lambda_x \text{cost}(a_t)) \right],$$

where the term $\lambda_x \text{cost}(a_t)$ penalizes the computational or operational cost associated with different action granularities (e.g., token length or execution time).

The central challenge lies in the fact that different tasks, and even different phases within the same task, naturally favor different action spaces. Naive training on a unified space often fails to develop reliable, context-aware selection behavior. To address this, we introduce a phased training curriculum that progressively builds this capability. Beginning with supervised initialization, proceeding to strengthen local preferences through single-turn optimization, and culminating in long-horizon reasoning via multi-turn reinforcement learning, our framework equips the model with robust, dynamic action-space selection capabilities.

The overall training pipeline is illustrated in Figure 2. We detail each stage in the following subsections.

3.1. Stage-1: Mixed-Space SFT

In the initial stage, we establish the model’s foundational capability to execute actions across heterogeneous spaces. We employ Supervised Fine-Tuning (SFT) on a balanced dataset comprising trajectories drawn from multiple action subspaces. By unifying data from these diverse modalities, the model learns to decode and generate valid actions within a composite action space. The primary objective of this stage is to construct a robust base model capable of grounding instructions into executable actions from diverse sources, prior to introducing complex decision-making tasks.

It is important to note that the resulting model, denoted as M_{mix} , does not yet autonomously select the optimal action space. Instead, it focuses on learning the syntax and semantics of multiple action types from the mixed data representation. The key challenge addressed here is enabling the model to interpret and generate actions for different spaces without modal interference, thereby laying a solid foundation for the adaptive decision-making mechanisms introduced in subsequent stages.

3.2. Stage-2: Single-Turn RL

In this stage, we empower the model to autonomously select the most appropriate action space using **Single-Turn Reinforcement Learning (STRL)**.

Warm-up: Diversity-Enhanced SFT. Standard datasets typically lack explicit annotations indicating which action space is optimal for a given context. To bridge this gap, we first perform a warm-up phase to expose the model to multiple viable paths. We construct prompts that encourage the model to generate candidate actions across all available action spaces. These candidates are filtered via a **rejection sampling** approach: only actions that successfully

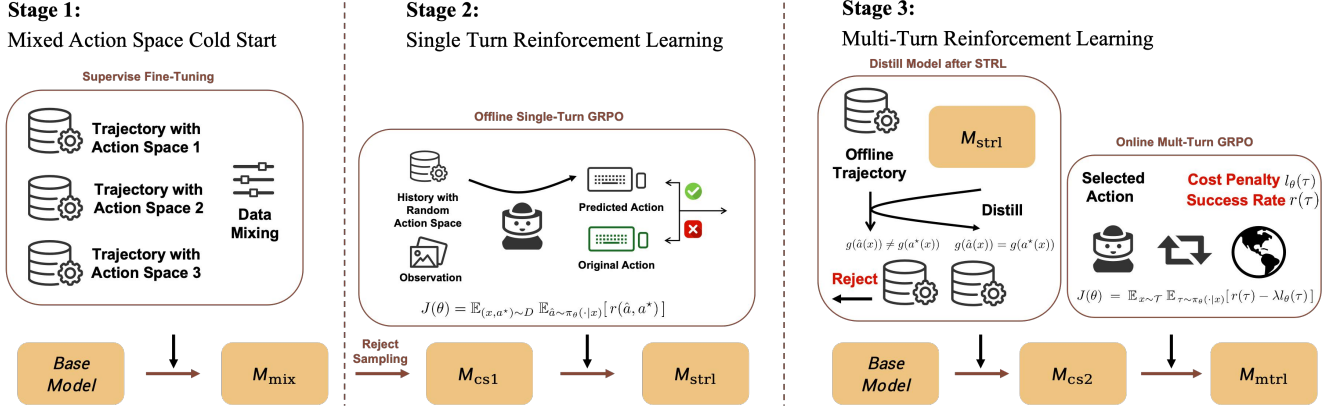


Figure 2. **Overview of the CrossHA Training Pipeline.** The pipeline comprises three distinct stages: Cold-Start Supervised Fine-Tuning (SFT), Single-Turn Reinforcement Learning (STRL), and Multi-Turn Reinforcement Learning (MTRL). In the first stage, the model learns to decode actions from a heterogeneous action space using a balanced dataset. During STRL, the model is fine-tuned to autonomously select the appropriate action space based on the immediate task context. Finally, in the MTRL stage, the policy is further optimized to balance task success rate with execution efficiency over long horizons. This progressive pipeline ensures CrossHA effectively adapts its action granularity across a wide range of tasks.

execute the task (verified by the environment or a parser) are retained as ground-truth annotations. We then fine-tune the model on this rebalanced, multi-space dataset. The resulting model, denoted as M_{cs1} , acquires the capability to generate valid actions in various formats. However, empirical observation reveals that while M_{cs1} can operate across spaces, its selection strategy remains stochastic rather than strategic—it does not yet deliberately optimize the interface choice for task efficiency or robustness.

Optimization via GRPO. To transition from stochastic capability to strategic selection, we employ Group Relative Policy Optimization (GRPO) [38], a reinforcement learning algorithm distinguished by its stability and resource efficiency. Unlike traditional methods like PPO that rely on a separate value function network for advantage estimation, GRPO utilizes the group statistics of multiple sampled outputs to estimate the baseline, thereby simplifying the optimization process.

Formally, for each query q , a group of outputs $\{o_1, o_2, \dots, o_G\}$ is sampled from the current policy π_{old} . The policy π_θ is then optimized via the following objective:

$$J_{GRPO}(\theta) = \mathbb{E}_{q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{old}} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \min \left(\rho_{i,t} \hat{A}_{i,t}, \text{clip}(\rho_{i,t}, 1 - \epsilon, 1 + \epsilon) \hat{A}_{i,t} \right) - \beta D_{KL}[\pi_\theta || \pi_{ref}] \right] \quad (1)$$

where $\rho_{i,t} = \frac{\pi_\theta(o_i|q, o_{<t})}{\pi_{old}(o_i|q, o_{<t})}$ is the probability ratio. Crucially,

$\hat{A}_{i,t}$ represents the advantage estimate, which is calculated based on the relative performance of output o_i compared to the group mean, rather than a learned value function. This encourages the model to favor actions that outperform their peers within the same sampled group.

Since this stage focuses on immediate, one-step decision-making (viewing the task as a single-turn problem to optimize action space selection), we term it the STRL stage. To endow the cold-start model M_{cs1} with **action-space autonomy**, we cast each example in the dataset D as a one-step decision problem. Let $\{\mathcal{A}_k\}_{k=1}^K$ denote the set of string-level action spaces and $\mathcal{A} = \bigcup_k \mathcal{A}_k$. We utilize a deterministic parser $g : \mathcal{A} \rightarrow \mathcal{R}$ that maps an action string to its canonical raw representation in $\mathcal{R}$. The reward function is defined as:

$$r(\hat{a}, a^*) = \mathbb{1}\{g(\hat{a}) = g(a^*)\}, \quad (2)$$

This reward is **action-space agnostic**: credit is granted whenever the parsed raw action matches the ground truth, regardless of the surface form (action space) used to generate $\hat{a}$. The STRL objective is to maximize the expected reward:

$$J(\theta) = \mathbb{E}_{(x, a^*) \sim D} \mathbb{E}_{\hat{a} \sim \pi_\theta(\cdot|x)} [r(\hat{a}, a^*)]. \quad (3)$$

Through this process, the model learns to ignore prior biases and autonomously select whichever action space most reliably yields the correct raw action for a given input. We denote the resulting model as M_{strl} .

3.3. Stage-3: Multi-Turn RL

Although M_{strl} achieves high accuracy in single-step action prediction, its probability distribution tends to be ex-

cessively peaked around specific action spaces, which limits exploration and hinders long-horizon task success. To explicitly cultivate robust, trajectory-level action-space selection, we adopt **Multi-Turn Reinforcement Learning (MTRL)**. In this stage, the optimization signal is derived from the episodic success rate, guiding the model to refine its policy for maximizing overall task completion rather than just immediate correctness.

Initialization via Self-Training (M_{cs2}). To accelerate training, we first distill the action-space preferences of M_{strl} back into a supervised format. We perform inference over the initial dataset D using M_{strl} . For each example (x, a^*) , let $\hat{a}$ denote the model’s prediction and let $g(\cdot)$ map an action string to its canonical raw representation. We construct a relabeled dataset D_{strl} via the following rule:

$$a'(x) = \begin{cases} \hat{a}(x), & \text{if } g(\hat{a}(x)) = g(a^*(x)), \\ a^*(x), & \text{otherwise.} \end{cases} \quad (4)$$

Essentially, if the predicted action is semantically consistent with the ground truth (as verified by g), we replace the original label with the action space chosen by M_{strl} ; otherwise, we retain the original label. This relabeling preserves the raw-action semantics while aligning the surface-level action space with the model’s learned preferences. We then fine-tune the base model on this relabeled dataset D_{strl} , following the same protocol as the previous SFT stage. We denote the resulting model as M_{cs2} . Since the ground truth in D_{strl} reflects the optimized choices of M_{strl} , M_{cs2} initializes the MTRL stage with a strong prior for appropriate action-space selection.

Trajectory Optimization. We subsequently fine-tune M_{cs2} using multi-turn RL on a curated task set $\mathcal{T}$. Given an instruction $x \in \mathcal{T}$, the policy $\pi_\theta(\cdot | x)$ interacts with the environment to generate a trajectory τ . We utilize a binary episodic reward:

$$r(\tau) = \mathbb{1}\{\text{success}(\tau)\} \quad (5)$$

The objective is to maximize the task success rate while penalizing excessive generation costs:

$$J(\theta) = \mathbb{E}_{x \sim \mathcal{T}} \mathbb{E}_{\tau \sim \pi_\theta(\cdot | x)} [r(\tau) - \lambda l_\theta(\tau)]. \quad (6)$$

where $l_\theta(\tau)$ denotes the total number of tokens produced by the model throughout the trajectory. This penalty term encourages the model to prefer concise action spaces (e.g., high-level APIs) over verbose ones (e.g., raw primitive commands) when both yield success. A detailed discussion of the specific GRPO implementation for this stage is provided in [Appendix C](#).

By optimizing [Equation 6](#), the model improves its capacity for dynamic, context-aware action space adaptation over long horizons. We denote the final model as M_{mtrl} , which serves as our final **CrossHA**.

4. Experiments

We present a comprehensive evaluation to validate our framework, focusing on how heterogeneous action spaces affect performance in open-ended environments. We describe the experimental setup, including training protocols, benchmarks, and evaluation metrics, and address the following questions:

Q1: Action Space Sensitivity. How does the choice of a fixed action space impact performance, and does the optimal space vary across task categories?

Q2: Benefits of Dynamic Switching. Does dynamic action-space selection outperform static baselines?

Q3: Generalization and Robustness. Does the model generalize to unseen tasks after multi-turn RL fine-tuning on a limited set of tasks?

4.1. Experimental Setup

Benchmark. We utilize Minecraft (version 1.16.5) as our primary evaluation environment [19]. The agent’s observation space consists exclusively of first-person RGB images with a resolution of $360 \times 640 \times 3$. The action space employs a discrete, human-like interface, comprising mouse movements, clicks, and keyboard commands. Further details regarding the observation and action spaces are provided in [Appendix A](#).

To assess our agent, we employ the OpenHA benchmark suite [49], which features over 800 tasks. All tasks are manually designed and verified, ensuring both richness and feasibility. We categorize these tasks into three groups based on the primary skills required: 1) **Mine Blocks**: Tasks involving navigation and physical interaction within the environment (e.g., locating and chopping down a specific type of tree); 2) **Craft Items**: Tasks requiring complex interactions with graphical user interfaces (GUIs), such as crafting at a table or smelting ores in a furnace; 3) **Kill Entities**: Tasks focusing on survival and combat, requiring the agent to engage with dynamic mobs.

In our evaluation protocol, we adopt two primary metrics: **Finished Tasks (FT)** and **Average Success Rate (ASR)**. FT measures the proportion of distinct tasks within a category for which the agent achieves at least one successful completion, reflecting the agent’s *task coverage* and generalization range. ASR quantifies the mean success rate across all tasks in the category, capturing *reliability* by accounting for the consistency of the agent’s performance on each specific task.

Table 1. The evaluation results of Minecraft agents across more than 800 tasks are presented. For each task category, we report three metrics: the success rate of a representative task (indicated by its icon), the percentage of tasks that the agent succeeds at least once (FT), and the average success rate across all tasks in the category (ASR) with standard deviation. Results highlighted in blue correspond to the second-best performances, while those in red represent the state-of-the-art performance for each metric across all agents.

Method	Mine Blocks			Kill Entities			Craft Items			All Tasks	
		FT ↑	ASR ↑		FT ↑	ASR ↑		FT ↑	ASR ↑	FT ↑	ASR ↑
Instruction-Conditioned Policies											
VPT [4]	20.0	30.7	6.0±11.4	10.0	24.6	3.6±7.7	0.0	6.7	0.8±3.3	20.7	3.5±8.4
STEVE-I [26]	50.0	29.4	8.0±17.0	0.0	14.7	3.9±12.0	0.0	16.4	3.2±8.4	20.2	5.0±12.4
ROCKET-1 [11]	60.0	57.5	18.9±24.3	60.0	63.9	27.9±29.3	0.0	0.0	0.0±0.0	45.5	15.6±24.9
JARVIS-VLA [25]	55.0	55.3	30.0±35.4	60.0	61.9	18.5±22.7	40.0	74.3	25.1±23.9	63.8	24.5±28.4
VLM-based Agents											
TextHA [49]	60.0	36.3	27.2±38.2	0.0	19.2	8.7±23.7	50.0	43.9	26.0±35.2	33.1	20.6±34.0
GroundingHA [11]	90.0	61.0	37.1±38.5	50.0	90.1	26.5±23.4	15.0	27.5	6.7±10.8	59.5	23.4±29.6
UI-TARS-1.5 [36]	-	-	42.1±20.4	-	-	31.0±16.4	0	0	36.7±17.2	-	33.8
MotionHA [6]	70.0	51.0	27.4±35.2	20.0	29.5	4.3±10.8	0.0	0.0	0.0±0.0	29.8	10.6±24.4
LanguageHA [14]	60.0	31.3	11.3±14.5	0.0	12.8	6.5±9.3	5.0	19.3	6.3±9.2	21.1	8.0±11.5
LatentHA [47]	70.0	54.2	24.4±31.1	50.0	24.6	8.5±17.9	0.0	19.1	3.0±7.5	32.6	12.0±23.0
OpenHA [49]	80.0	67.3	30.1±13.9	70.0	62.6	32.5±9.2	80.0	58.8	31.9±13.7	62.8	31.5±12.5
Game-TARS [50]	-	-	50.14±20.7	-	-	38.1±24.6	-	-	39.1±27.5	-	42.2
Ours											
CrossHA(w/o STRL stage)	63.6	35.1	39.0±46.5	28.6	43.9	27.7±43.9	57.1	55.7	58.0±48.4	44.9	41.6±47.9
CrossHA	94.7	45.2	40.0±48.3	66.6	58.1	45.1±43.5	83.3	72.7	78.8±41.0	58.7	54.6±47.6

Training Setup. 1) **Model Initialization.** We initialize our training with Qwen2-VL-7B-Instruct [44], a pre-trained vision-language model demonstrating strong multimodal understanding. We fine-tune this model using diverse Minecraft-specific VQA and captioning datasets [25], yielding the base model M_{base} . 2) **Dataset Construction.** We generate grounding and motion ground-truth annotations by integrating two systems: a SAM-based [35] grounding pipeline and a motion-generation module built upon a fine-tuned MineCLIP [15] model. These systems annotate both VPT data and a subset of contractor-collected trajectories. Since suitable grounding or motion annotations cannot be generated for every trajectory, the resulting action space distribution is imbalanced. These annotated data are integrated to form the mixed-action-space dataset D_{mix} .

We then process M_{base} and D_{mix} through the pipeline described in section 3. During the MTRL stage, we select 10 tasks from each of the three primary OpenHA categories: `craft_item`, `kill_entity`, and `mine_block`. This results in a total of 30 training tasks. For online RL, we train using GRPO for over 80 iterations, with each iteration involving over 6,400 environment interactions. This

extensive training regime allows the model’s performance to converge to a stable, high-level policy.

Baselines. Our analysis compares CrossHA against variants of the base model M_{base} fine-tuned (via either SFT or RL) on datasets restricted to single-type action spaces. We categorize these baselines based on their action space formats: LanguageHA, GroundingHA, MotionHA, RawHA, and LatentHA. We also include several representative LLM-based agents, such as Jarvis-VLA [25], and hierarchical agents fine-tuned with SFT on fixed action spaces. In addition, we consider recent unified agent frameworks such as OpenHA [49], which integrate multiple action spaces within a hierarchical agent architecture and enable flexible switching during execution. However, such switching behavior is not explicitly optimized as a learnable decision variable, in contrast to CrossHA, which formulates action-space selection as a policy learning problem and optimizes it via reinforcement learning. For broader context, we further compare against specialized policies trained on the VPT dataset, including the original VPT [4], ROCKET-1 [11], and STEVE-I [26], as well as Game-TARS [50], a

large-scale pretrained game agent that leverages multimodal pretraining and hierarchical control for general gameplay.

4.2. Main Results

As presented in Table 1, CrossHA achieves state-of-the-art performance across all three major task categories. It substantially outperforms existing instruction-conditioned policies and hierarchical agents on overall evaluation metrics. Our analysis highlights three key observations:

Clear Advantages Over Single-Action-Space Agents.

Agents confined to a single action space exhibit strong but narrow specialization, leading to pronounced performance asymmetries across task types. For instance, GroundingHA excels at *Kill Entity* tasks (FT: 90.1%), where visual tracking and sustained attention are essential. Conversely, MotionHA performs better on *Mine Block* tasks that emphasize locomotion, while RawHA demonstrates advantages in *Craft Item* tasks due to its fine-grained control capabilities. However, these advantages are domain-specific and fail to generalize across categories—none of the single-action-space baselines achieve uniformly strong results. This underscores the inherent limitations of rigid, action-space-specific designs.

RL Facilitates Robust OOD Generalization. Comparing SFT-only agents (e.g., OpenHA, MotionHA) with their RL-enhanced counterparts (e.g., CrossHA variants) reveals consistent and substantial gains in both ASR and FT. Remarkably, although RL fine-tuning is conducted on only 30 tasks, the improvements generalize effectively to over 800 evaluation tasks, many of which differ significantly from the training distribution. These gains are particularly pronounced in fine-grained control domains such as *Craft Item*, where trajectory optimization offers a distinct advantage. This confirms that reinforcement learning, when applied atop large pre-trained VLA models, serves as a critical component for achieving out-of-distribution (OOD) generalization in scalable embodied agents.

CrossHA Achieves Balanced, Omni-Category Performance. Unlike prior hierarchical or modular agents, CrossHA achieves the best or second-best results in nearly every metric across all categories. It demonstrates a unique ability to balance proficiency across *Mine Blocks*, *Kill Entities*, and *Craft Items*—a versatility not observed in prior baselines. Furthermore, it attains high peak success rates (e.g., 94.7% in *Mine Blocks*, 83.3% in *Craft Items*) without sacrificing generality. These results strongly suggest that CrossHA effectively learns to coordinate and select the most suitable action space based on task context, rather than over-specializing in a single mode. This dynamic switching mechanism is key to its robust performance.

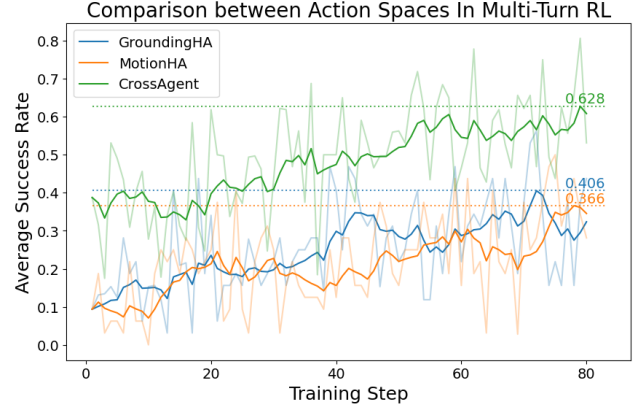


Figure 3. **Performance Comparison Across Action Spaces.** The heterogeneous action space of CrossHA enables superior data efficiency and higher asymptotic performance during multi-turn reinforcement learning, compared to single-space baselines.

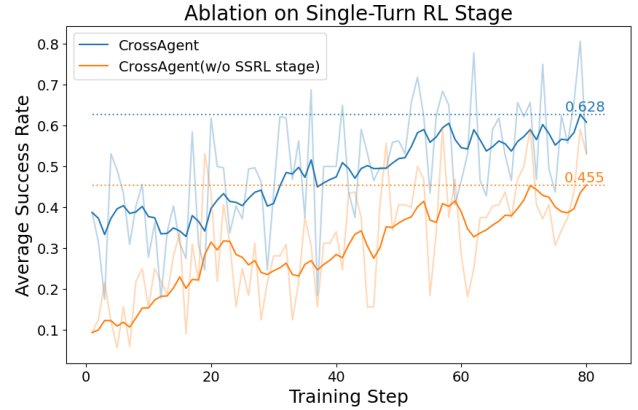


Figure 4. **Effect of the Single-Turn RL (STRL) Stage.** Training curves comparing CrossHA with and without the STRL phase. The inclusion of STRL significantly enhances training efficiency and accelerates convergence in the subsequent MTRL stage, despite its low computational cost.

4.3. Ablation Studies

Impact of Mixed Action Spaces. To evaluate the benefit of a unified heterogeneous action space, we compare CrossHA with single-space baselines, *GroundingHA* and *MotionHA*. These baselines are constructed by initializing the MTRL stage with only the grounding-space or motion-space subset of D_{strl} , respectively, while CrossHA uses the full mixed dataset. All models undergo the same 200-step SFT cold start before MTRL, with no repeated samples during this phase.

As shown in Figure 3, CrossHA demonstrates significantly faster convergence and higher asymptotic performance compared to the single-space baselines. This indicates that the capacity to autonomously and dynamically se-

Table 2. Evaluation results of RL agents on In-Distribution (ID) and Out-of-Distribution (OOD) tasks. We report the success rate and standard deviation. **Red** indicates the best performance, and **Blue** indicates the second best.

Method	In-Distribution Evaluation				Out-of-Distribution Evaluation			
	Mine Blocks	Kill Entities	Craft Items	All Tasks	Mine Blocks	Kill Entities	Craft Items	All Tasks
RawHA-RL	65.2 $\pm$ 32.1	41.6 $\pm$ 21.8	96.2$\pm$3.8	70.1$\pm$33.6	28.9 $\pm$ 44.2	28.4$\pm$32.3	69.8 $\pm$ 44.4	42.4 $\pm$ 45.1
GroundingHA-RL	46.8 $\pm$ 35.6	48.8$\pm$34.5	69.3 $\pm$ 28.4	52.6 $\pm$ 31.5	27.2 $\pm$ 42.4	33.9$\pm$35.9	57.2 $\pm$ 48.1	39.4 $\pm$ 44.3
MotionHA-RL	78.6$\pm$24.6	44.0 $\pm$ 25.2	63.0 $\pm$ 20.7	61.9 $\pm$ 27.5	41.5$\pm$45.5	26.8 $\pm$ 26.7	49.0 $\pm$ 47.3	39.1 $\pm$ 42.0
CrossHA(w/o STRL)	63.3 $\pm$ 28.9	24.0 $\pm$ 26.4	76.1 $\pm$ 27.2	54.5 $\pm$ 35.3	39.0 $\pm$ 46.5	22.2 $\pm$ 42.6	58.0 $\pm$ 48.4	39.7 $\pm$ 48.1
CrossHA	70.7$\pm$33.3	52.1$\pm$22.9	83.7$\pm$25.5	68.8$\pm$30.5	40.0$\pm$48.3	28.4$\pm$33.2	78.8$\pm$41.0	49.1$\pm$46.6

lect among multiple action spaces affords the model greater flexibility. When confronting diverse and complex task dynamics, this flexibility translates into higher success rates and more robust task completion, thereby significantly enhancing the data efficiency of RL.

Necessity of the STRL Stage. We further study the role of the Single-Turn RL (STRL) stage. For the variant *without* STRL, we skip Stage 2 and initialize MTRL with the original balanced dataset D_{bal} instead of the STRL-reabeled dataset D_{strl} . We then apply the same MTRL training to both models.

As shown in Figure 4, including STRL improves both sample efficiency and final performance. This indicates that STRL provides an effective warm start for action-space selection, enabling more efficient exploration and more reliable action sequencing during the expensive MTRL stage. Results in Table 1 further show that CrossHA with STRL achieves higher success rates across a broader range of tasks, demonstrating stronger cross-task generalization.

4.4. Generalization Evaluation

While Table 1 shows strong OOD results, an important question is whether reinforcement learning harms generalization. To examine this, we compare performance on In-Distribution (ID) training tasks and OOD evaluation tasks in Table 2.

Robust Performance on OOD Tasks. CrossHA achieves the best success rate on *Craft Items* (78.8%) and strong performance on *Mine Blocks* (40.0%) and *All Tasks* (49.1%), indicating better generalization than other RL baselines. CrossHA (w/o STRL) remains competitive but consistently underperforms the full model, especially on *Craft Items* (58.0% vs. 78.8%) and *All Tasks* (39.7% vs. 49.1%), showing that STRL is important for avoiding sub-optimal policies and improving generalization. In contrast, single-space agents such as RawHA-RL, GroundingHA-RL, and MotionHA-RL show larger variance and weaker transfer to tasks outside their specialized action spaces.

Analysis of the Generalization Gap (ID vs. OOD).

Comparing ID and OOD results reveals different generalization behaviors. Baselines such as RawHA-RL and MotionHA-RL achieve very high ID performance (e.g., 96.2% for RawHA-RL on *Craft Items*) but drop sharply on OOD tasks, suggesting stronger overfitting. In contrast, CrossHA performs strongly on ID tasks (83.7% on *Craft Items* and 68.8% overall) while maintaining a smaller generalization gap. This suggests that dynamic action-space selection helps mitigate overfitting and better transfers learned environment knowledge to unseen tasks.

Conclusion on RL Fine-Tuning. The results underscore a key insight: the benefits of RL fine-tuning, especially when combined with heterogeneous action spaces, far outweigh the risks of overfitting. CrossHA sets a new standard for OOD generalization, establishing that a unified agent capable of dynamic interface switching is more robust than specialized experts. The combination of large-scale pre-training, the STRL warm-up, and MTRL optimization enables the agent to master complex distributions, proving that reinforcement learning is a viable path for building general-purpose open-world agents.

5. Conclusion

In this work, we present CrossHA, a unified agentic model that integrates heterogeneous action spaces from high-level scripts to low-level atomic commands within a single policy. Through a training pipeline based on Single- and Multi-Turn GRPO, the model learns to dynamically select the most suitable action space at each step.

Extensive evaluations on over 800 Minecraft tasks demonstrate that CrossHA achieves state-of-the-art performance, significantly outperforming baselines with fixed action spaces. These results highlight the importance of treating action-space selection as a learnable component for improving performance and generalization.

Future work includes improving the efficiency of multi-turn RL and extending the framework to real-world robotics settings, where additional challenges such as safety and latency arise.

Acknowledgments

This work was supported by the National Science and Technology Major Project (2022ZD0114902).

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023. 1
- [2] Anthropic. Introducing the model context protocol, 2024. 1, 2
- [3] Anthropic. Introducing claude 4, 2025. 1, 2, 3
- [4] Bowen Baker, Ilge Akkaya, Peter Zhokov, Joost Huizinga, Jie Tang, Adrien Ecoffet, Brandon Houghton, Raul Sampeiro, and Jeff Clune. Video pretraining (vpt): Learning to act by watching unlabeled online videos. *Advances in Neural Information Processing Systems*, 35:24639–24654, 2022. 6, 12
- [5] Suneel Belkhale, Tianli Ding, Ted Xiao, Pierre Sermanet, Quon Vuong, Jonathan Tompson, Yevgen Chebotar, Debidda Dwibedi, and Dorsa Sadigh. Rt-h: Action hierarchies using language. *arXiv preprint arXiv:2403.01823*, 2024. 1
- [6] Suneel Belkhale, Tianli Ding, Ted Xiao, Pierre Sermanet, Quon Vuong, Jonathan Tompson, Yevgen Chebotar, Debidda Dwibedi, and Dorsa Sadigh. Rt-h: Action hierarchies using language. *arXiv preprint arXiv:2403.01823*, 2024. 1, 2, 6, 13
- [7] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Jasmine Hsu, et al. Rt-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2022. 1
- [8] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023. 2
- [9] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023. 1
- [10] Qingwen Bu, Yanting Yang, Jisong Cai, Shenyuan Gao, Guanghui Ren, Maoqing Yao, Ping Luo, and Hongyang Li. Univla: Learning to act anywhere with task-centric latent actions. *arXiv preprint arXiv:2505.06111*, 2025. 3
- [11] Shaofei Cai, Zihao Wang, Kewei Lian, Zhancun Mu, Xiaojian Ma, Anji Liu, and Yitao Liang. Rocket-1: Mastering open-world interaction with visual-temporal context prompting. *arXiv preprint arXiv:2410.17856*, 2024. 6, 13
- [12] Jingwen Deng, Zihao Wang, Shaofei Cai, Anji Liu, and Yitao Liang. Open-world skill discovery from unsegmented demonstrations. *arXiv preprint arXiv:2503.10684*, 2025. 13
- [13] Xiang Deng, Kelvin Guu, Panupong Pasupat, Afra Akyürek, Sheng Zhuang, Wenlong Chen, Tatsunori Hashimoto, Kelvin Guu, and Percy Liang. Mind2web: Towards a generalist agent for the web. In *NeurIPS Datasets and Benchmarks*, 2023. 2
- [14] Danny Driess, Fei Xia, Mehdi SM Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, et al. Palme: An embodied multimodal language model. *arXiv preprint arXiv:2303.03378*, 2023. 1, 2, 6, 13
- [15] Linxi Fan, Guanzhi Wang, Yunfan Jiang, Ajay Mandlekar, Yuncong Yang, Haoyi Zhu, Andrew Tang, De-An Huang, Yuke Zhu, and Anima Anandkumar. Minedojo: Building open-ended embodied agents with internet-scale knowledge. *Advances in Neural Information Processing Systems Datasets and Benchmarks*, 2022. 2, 6
- [16] Jiazhan Feng, Shijue Huang, Xingwei Qu, Ge Zhang, Yujia Qin, Baoquan Zhong, Chengquan Jiang, Jinxin Chi, and Wanjun Zhong. Retool: Reinforcement learning for strategic tool use in llms. *arXiv preprint arXiv:2504.11536*, 2025. 1, 2
- [17] Dong Guo, Faming Wu, Feida Zhu, Fuxing Leng, Guang Shi, Haobin Chen, Haoqi Fan, Jian Wang, Jianyu Jiang, Jiawei Wang, et al. Seed1. 5-vl technical report. *arXiv preprint arXiv:2505.07062*, 2025. 1
- [18] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025. 2
- [19] William H Guss, Brandon Houghton, Nicholay Topin, Phillip Wang, Cayden Codel, Manuela Veloso, and Ruslan Salakhutdinov. Miner1: A large-scale dataset of minecraft demonstrations. *arXiv preprint arXiv:1907.13440*, 2019. 5, 12
- [20] Yuxuan Huang, Yihang Chen, Haozheng Zhang, Kang Li, Huichi Zhou, Meng Fang, Linyi Yang, Xiaoguang Li, Lifeng Shang, Songcen Xu, et al. Deep research agents: A systematic examination and roadmap. *arXiv preprint arXiv:2506.18096*, 2025. 1
- [21] Hongrui Jia, Jitong Liao, Xi Zhang, Haiyang Xu, Tianbao Xie, Chaoya Jiang, Ming Yan, Si Liu, Wei Ye, and Fei Huang. Osvorld-mcp: Benchmarking mcp tool invocation in computer-use agents. *arXiv preprint arXiv:2510.24563*, 2025. 3
- [22] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024. 2, 13
- [23] Jason Lee, Jiafei Duan, Haoquan Fang, Yuquan Deng, Shuo Liu, Boyang Li, Bohan Fang, Jieyu Zhang, Yi Ru Wang, Sangho Lee, et al. Molmoact: Action reasoning models that can reason in space. *arXiv preprint arXiv:2508.07917*, 2025. 13
- [24] Kaixin Li, Ziyang Meng, Hongzhan Lin, Ziyang Luo, Yuchen Tian, Jing Ma, Zhiyong Huang, and Tat-Seng

- Chua. Screenspot-pro: Gui grounding for professional high-resolution computer use. *arXiv preprint arXiv:2504.07981*, 2025. 2
- [25] Muyao Li, Zihao Wang, Kaichen He, Xiaojian Ma, and Yitao Liang. Jarvis-vla: Post-training large-scale vision language models to play visual games with keyboards and mouse. *arXiv preprint arXiv:2503.16365*, 2025. 3, 6
- [26] Shalev Lifshitz, Keiran Paster, Harris Chan, Jimmy Ba, and Sheila McIlraith. Steve-1: A generative model for text-to-behavior in minecraft. *Advances in Neural Information Processing Systems*, 36, 2024. 6, 13
- [27] Fanqi Lin, Ruiqian Nai, Yingdong Hu, Jiacheng You, Junming Zhao, and Yang Gao. Onetwovla: A unified vision-language-action model with adaptive reasoning. *arXiv preprint arXiv:2505.11917*, 2025. 3
- [28] Haowei Lin, Zihao Wang, Jianzhu Ma, and Yitao Liang. Mcu: A task-centric framework for open-ended agent evaluation in minecraft. *arXiv preprint arXiv:2310.08367*, 2023. 2
- [29] Abby O’Neill, Abdul Rehman, Abhinav Gupta, Abhiram Maddukuri, Abhishek Gupta, Abhishek Padalkar, Abraham Lee, Acorn Pooley, Agrim Gupta, Ajay Mandlekar, et al. Open x-embodiment: Robotic learning datasets and rt-x models. *arXiv preprint arXiv:2310.08864*, 2023. 2
- [30] OpenAI. Chatgpt: Optimizing language models for dialogue, 2023. 1, 3
- [31] OpenAI. Introducing chatgpt agent: bridging research and action, 2025. 1
- [32] OpenAI. Introducing deep research, 2025. 1, 3
- [33] openai. Operator, 2025. 1, 3
- [34] Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, et al. Ui-tars: Pioneering automated gui interaction with native agents. *arXiv preprint arXiv:2501.12326*, 2025. 1
- [35] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, et al. Sam 2: Segment anything in images and videos. *arXiv preprint arXiv:2408.00714*, 2024. 6
- [36] ByteDance Seed. Ui-tars-1.5. <https://seed-tars.com/1.5>, 2025. 6
- [37] Nur Muhammad Shafiullah, Zichen Cui, Ariuntuya Arty Altanzaya, and Lerrel Pinto. Behavior transformers: Cloning k modes with one stone. *Advances in neural information processing systems*, 35:22955–22968, 2022. 13
- [38] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. 2, 4
- [39] Linxin Song, Yutong Dai, Viraj Prabhu, Jieyu Zhang, Taiwei Shi, Li Li, Junnan Li, Silvio Savarese, Zeyuan Chen, Jieyu Zhao, Ran Xu, and Caiming Xiong. Coact-1: Computer-using agents with coding as actions. *arXiv preprint arXiv:2508.03923*, 2025. 3
- [40] Gemini Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, Katie Millican, et al. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023. 1, 3
- [41] Seed TARS Team. Ui-tars-2 technical report: Advancing gui agent with multi-turn reinforcement learning. *arXiv preprint arXiv: 2509.02544*, 2025. 1, 2, 3
- [42] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023. 1
- [43] Aaron Van Den Oord, Oriol Vinyals, et al. Neural discrete representation learning. *Advances in neural information processing systems*, 30, 2017. 13
- [44] Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, et al. Qwen2-vl: Enhancing vision-language model’s perception of the world at any resolution. *arXiv preprint arXiv:2409.12191*, 2024. 1, 6, 14
- [45] Zihao Wang, Shaofei Cai, Guanzhou Chen, Anji Liu, Xiaojian Ma, Yitao Liang, and Team CraftJarvis. Describe, explain, plan and select: interactive planning with large language models enables open-world multi-task agents. In *Proceedings of the 37th International Conference on Neural Information Processing Systems*, pages 34153–34189, 2023. 13
- [46] Zihao Wang, Shaofei Cai, Anji Liu, Yonggang Jin, Jinhong Hou, Bawei Zhang, Haowei Lin, Zhaofeng He, Zilong Zheng, Yaodong Yang, et al. Jarvis-1: Open-world multi-task agents with memory-augmented multimodal language models. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024. 3
- [47] Zihao Wang, Shaofei Cai, Zhancun Mu, Haowei Lin, Ceyao Zhang, Xuejie Liu, Qing Li, Anji Liu, Xiaojian Ma, and Yitao Liang. Omnijarvis: Unified vision-language-action tokenization enables open-world instruction following agents. *Advances in Neural Information Processing Systems*, 2024. 6
- [48] Zihao Wang, Anji Liu, Haowei Lin, Jiaqi Li, Xiaojian Ma, and Yitao Liang. Rat: Retrieval augmented thoughts elicit context-aware reasoning in long-horizon generation. *arXiv preprint arXiv:2403.05313*, 2024. 2
- [49] Zihao Wang, Muyao Li, Kaichen He, Xiangyu Wang, Zhancun Mu, Anji Liu, and Yitao Liang. Openha: A series of open-source hierarchical agentic models in minecraft. *arXiv preprint arXiv:2509.13347*, 2025. 3, 5, 6, 14
- [50] Zihao Wang, Xujing Li, Yining Ye, Junjie Fang, Haoming Wang, Longxiang Liu, Shihao Liang, Junting Lu, Zhiyong Wu, Jiazhan Feng, et al. Game-tars: Pretrained foundation models for scalable generalist multimodal game agents. *arXiv preprint arXiv:2510.23691*, 2025. 3, 6
- [51] Yiheng Xu, Zekun Wang, Junli Wang, Dunjie Lu, Tianbao Xie, Amrita Saha, Doyen Sahoo, Tao Yu, and Caiming Xiong. Aguvis: Unified pure vision agents for autonomous gui interaction. *arXiv preprint arXiv:2412.04454*, 2024. 2

- [52] Yunhe Yan, Shihe Wang, Jiajun Du, Yexuan Yang, Yuxuan Shan, Qichen Qiu, Xianqing Jia, Xinge Wang, Xin Yuan, Xu Han, Mao Qin, Yinxiao Chen, Chen Peng, Shangguang Wang, and Mengwei Xu. Mcpworld: A unified benchmarking testbed for api, gui, and hybrid computer use agents. *arXiv preprint arXiv:2506.07672*, 2025. [3](#)
- [53] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. [1](#)
- [54] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024. [2](#)
- [55] Haoqi Yuan, Zhancun Mu, Feiyang Xie, and Zongqing Lu. Pre-training goal-based models for sample-efficient reinforcement learning. In *The Twelfth International Conference on Learning Representations*, 2024. [13](#)
- [56] Yifan Zhong, Fengshuo Bai, Shaofei Cai, Xuchuan Huang, Zhang Chen, Xiaowei Zhang, Yuanfei Wang, Shaoyang Guo, Tianrui Guan, Ka Nam Lui, et al. A survey on vision-language-action models: An action tokenization perspective. *arXiv preprint arXiv:2507.01925*, 2025. [13](#)
- [57] Yifan Zhong, Xuchuan Huang, Ruochong Li, Ceyao Zhang, Yitao Liang, Yaodong Yang, and Yuanpei Chen. Dex-graspvla: A vision-language-action framework towards general dexterous grasping, 2025. [13](#)